TOPIC 3 : DIVIDE AND CONQUER

1.Maximum and Minimum in Array

main.py

```python
def find_min_max(arr):
    minimum = arr[0]
    maximum = arr[0]

    for i in range(1, len(arr)):
        if arr[i] < minimum:
            minimum = arr[i]
        if arr[i] > maximum:
            maximum = arr[i]

    return minimum, maximum

tests = [
    [5, 7, 3, 4, 9, 12, 6, 2],
    [1, 3, 5, 7, 9, 11, 13, 15, 17],
    [22, 34, 35, 36, 43, 67, 12, 13, 15, 17]
]

for arr in tests:
    minimum, maximum = find_min_max(arr)
    print("Input:", arr)
    print("Min =", minimum, "Max =", maximum)
    print()
```

Output

```
Input: [5, 7, 3, 4, 9, 12, 6, 2]
Min = 2 Max = 12

Input: [1, 3, 5, 7, 9, 11, 13, 15, 17]
Min = 1 Max = 17

Input: [22, 34, 35, 36, 43, 67, 12, 13, 15, 17]
Min = 12 Max = 67


=== Code Execution Successful ===
```

2. Min-Max in Sorted Array

main.py

```python
def find_min_max(arr):
    minimum = arr[0]
    maximum = arr[0]

    for i in range(1, len(arr)):
        if arr[i] < minimum:
            minimum = arr[i]
        if arr[i] > maximum:
            maximum = arr[i]

    return minimum, maximum

tests = [
    [2, 4, 6, 8, 10, 12, 14, 18],
    [11, 13, 15, 17, 19, 21, 23, 35, 37],
    [22, 34, 35, 36, 43, 67, 12, 13, 15, 17]
]

for arr in tests:
    minimum, maximum = find_min_max(arr)
    print("Input:", arr)
    print("Min =", minimum, "Max =", maximum)
    print()
```

Output

```
Input: [2, 4, 6, 8, 10, 12, 14, 18]
Min = 2 Max = 18

Input: [11, 13, 15, 17, 19, 21, 23, 35, 37]
Min = 11 Max = 37

Input: [22, 34, 35, 36, 43, 67, 12, 13, 15, 17]
Min = 12 Max = 67


=== Code Execution Successful ===
```

3. Merge Sort

main.py

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result += left[i:]
    result += right[j:]

    return result

tests = [
    [31, 23, 35, 27, 11, 21, 15, 28],
```

Output

```
Input : [31, 23, 35, 27, 11, 21, 15, 28]
Output: [11, 15, 21, 23, 27, 28, 31, 35]

Input : [22, 34, 25, 36, 43, 67, 52, 13, 65, 17]
Output: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]


=== Code Execution Successful ===
```

4. Merge Sort with Comparison Count

main.py

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr, 0

    mid = len(arr) // 2
    left, c1 = merge_sort(arr[:mid])
    right, c2 = merge_sort(arr[mid:])

    result = []
    i = j = 0
    count = c1 + c2

    while i < len(left) and j < len(right):
        count += 1

        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result += left[i:]
    result += right[j:]

    return result, count
```

Output

```
Input: [12, 4, 78, 23, 45, 67, 89, 1]
Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

Input: [38, 27, 43, 3, 9, 82, 10]
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
Comparisons: 13


=== Code Execution Successful ===
```

5. Quick Sort with First Element Pivot

main.py

```python
def quick_sort(arr, low, high):
    if low < high:
        pivot = arr[low]
        i = low
        j = high

        while i < j:
            while i < high and arr[i] <= pivot:
                i += 1

            while arr[j] > pivot:
                j -= 1

            if i < j:
                arr[i], arr[j] = arr[j], arr[i]

        arr[low], arr[j] = arr[j], arr[low]

        print("After partition:", arr)

        quick_sort(arr, low, j - 1)
        quick_sort(arr, j + 1, high)

tests = [
    [10, 16, 8, 12, 15, 6, 3, 9, 5],
    [12, 4, 78, 23, 45, 67, 89, 1],
```

Output

```
Input: [10, 16, 8, 12, 15, 6, 3, 9, 5]
After partition: [6, 5, 8, 9, 3, 10, 15, 12, 16]
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]
After partition: [3, 5, 6, 8, 9, 10, 15, 12, 16]
After partition: [3, 5, 6, 8, 9, 10, 12, 15, 16]
Sorted: [3, 5, 6, 8, 9, 10, 12, 15, 16]

Input: [12, 4, 78, 23, 45, 67, 89, 1]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 78, 89]
Sorted: [1, 4, 12, 23, 45, 67, 78, 89]

Input: [38, 27, 43, 3, 9, 82, 10]
After partition: [9, 27, 10, 3, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 43, 82]
Sorted: [3, 9, 10, 27, 38, 43, 82]


=== Code Execution Successful ===
```

6. Quick Sort with Middle Element Pivot

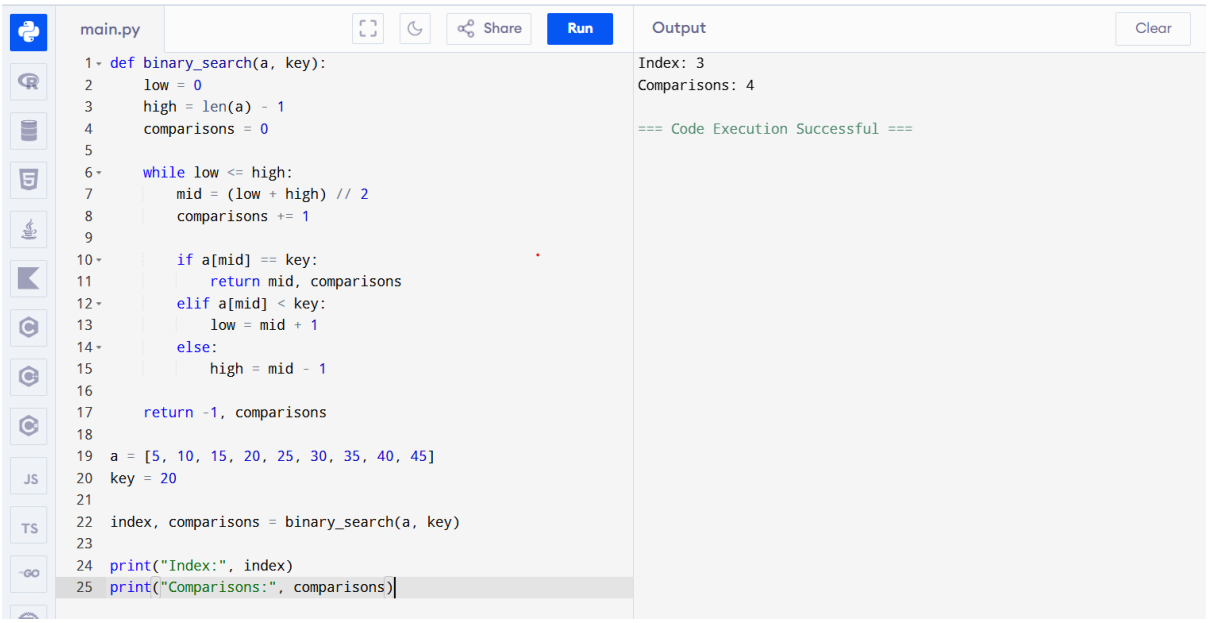
main.py

```python
def quick_sort(arr, low, high):
    if low < high:
        i = low
        j = high
        pivot = arr[(low + high) // 2]

        while i <= j:
            while arr[i] < pivot:
                i += 1

            while arr[j] > pivot:
                j -= 1

            if i <= j:
                arr[i], arr[j] = arr[j], arr[i]
                i += 1
                j -= 1

        print("After partition:", arr)

        if low < j:
            quick_sort(arr, low, j)

        if i < high:
            quick_sort(arr, i, high)
```

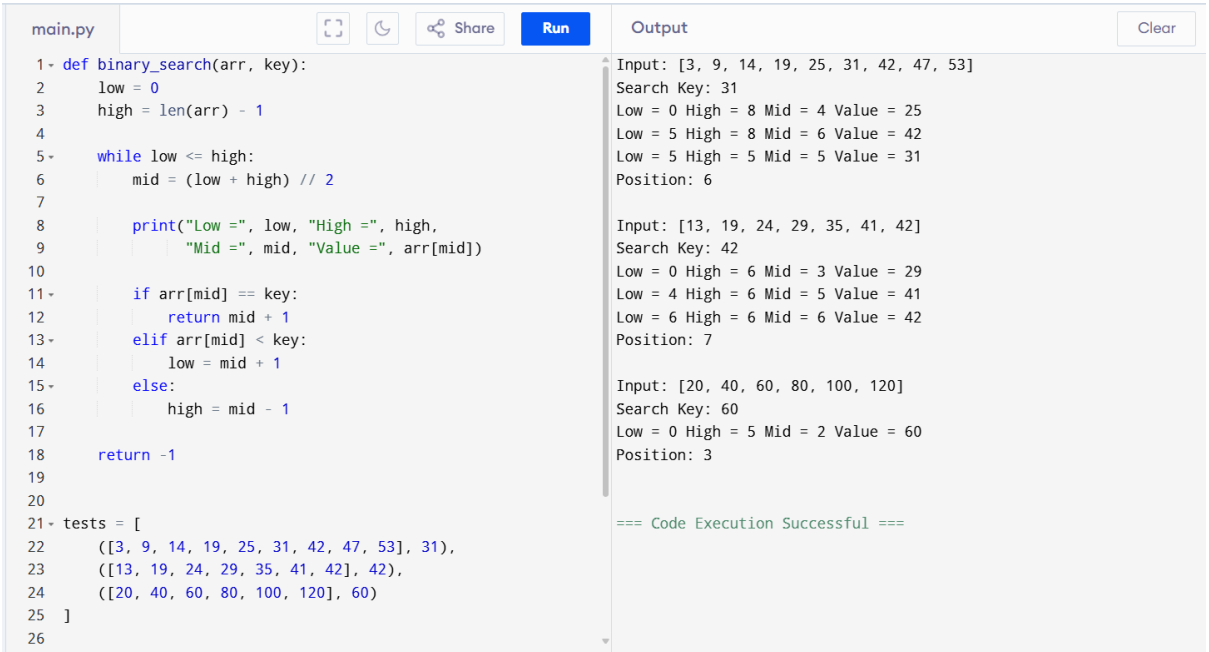
Output

```
After partition: [19, 22, 31, 35, 46, 58, 72, 91]
Sorted: [19, 22, 31, 35, 46, 58, 72, 91]

Input: [31, 23, 35, 27, 11, 21, 15, 28]
After partition: [15, 23, 21, 11, 27, 35, 31, 28]
After partition: [15, 11, 21, 23, 27, 35, 31, 28]
After partition: [11, 15, 21, 23, 27, 35, 31, 28]
After partition: [11, 15, 21, 23, 27, 28, 31, 35]
After partition: [11, 15, 21, 23, 27, 28, 31, 35]
Sorted: [11, 15, 21, 23, 27, 28, 31, 35]

Input: [22, 34, 25, 36, 43, 67, 52, 13, 65, 17]
After partition: [22, 34, 25, 36, 17, 13, 52, 67, 65, 43]
After partition: [22, 13, 17, 36, 25, 34, 52, 67, 65, 43]
After partition: [13, 22, 17, 36, 25, 34, 52, 67, 65, 43]
After partition: [13, 17, 22, 36, 25, 34, 52, 67, 65, 43]
After partition: [13, 17, 22, 25, 36, 34, 52, 67, 65, 43]
After partition: [13, 17, 22, 25, 34, 36, 52, 67, 65, 43]
After partition: [13, 17, 22, 25, 34, 36, 52, 43, 65, 67]
After partition: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]
After partition: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]
Sorted: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]


=== Code Execution Successful ===
```

7.Binary Search with Comparison Count



8. Binary Search and Mid-Point Analysis



9. K Closest Points

main.py Share Run Output Clear

```python
def k_closest(points, k):
    points.sort(key=lambda p: p[0] * p[0] + p[1] * p[1])
    return points[:k]


tests = [
    ([[1, 3], [-2, 2], [5, 8], [0, 1]], 2),
    ([[1, 3], [-2, 2]], 1),
    ([[3, 3], [5, -1], [-2, 4]], 2)
]

for i, (points, k) in enumerate(tests, 1):
    result = k_closest(points, k)
    print("Test Case", i)
    print("Input:", points)
    print("K =", k)
    print("Output:", result)
    print()
```

```
Test Case 1
Input: [[0, 1], [-2, 2], [1, 3], [5, 8]]
K = 2
Output: [[0, 1], [-2, 2]]

Test Case 2
Input: [[-2, 2], [1, 3]]
K = 1
Output: [[-2, 2]]

Test Case 3
Input: [[3, 3], [-2, 4], [5, -1]]
K = 2
Output: [[3, 3], [-2, 4]]


=== Code Execution Successful ===
```

10. 4-Sum Problem

main.py Share Run Output

```python
def four_sum_count(A, B, C, D):
    count = Counter()

    for a in A:
        for b in B:
            count[a + b] += 1

    answer = 0

    for c in C:
        for d in D:
            answer += count[-(c + d)]

    return answer


tests = [
    ([1, 2], [-2, -1], [-1, 2], [0, 2]),
    ([0], [0], [0], [0])
]

for i, (A, B, C, D) in enumerate(tests, 1):
    result = four_sum_count(A, B, C, D)

    print("Test Case", i)
```

```
Test Case 1
A = [1, 2]
B = [-2, -1]
C = [-1, 2]
D = [0, 2]
Output: 2

Test Case 2
A = [0]
B = [0]
C = [0]
D = [0]
Output: 1


=== Code Execution Successful ===
```

11.Median of Medians

main.py

Share Run

```python
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k - 1]

    groups = []

    for i in range(0, len(arr), 5):
        group = sorted(arr[i:i + 5])
        groups.append(group[len(group) // 2])

    pivot = median_of_medians(groups, (len(groups) + 1) // 2
        )

    smaller = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    larger = [x for x in arr if x > pivot]

    if k <= len(smaller):
        return median_of_medians(smaller, k)
    elif k <= len(smaller) + len(equal):
        return pivot
    else:
        return median_of_medians(
            larger,
            k - len(smaller) - len(equal)
        )
```

Output Clear

```
Test Case 1
Array: [12, 3, 5, 7, 19]
K = 2
K-th Smallest Element: 5

Test Case 2
Array: [12, 3, 5, 7, 4, 19, 26]
K = 3
K-th Smallest Element: 5

Test Case 3
Array: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
K = 6
K-th Smallest Element: 6


=== Code Execution Successful ===
```

12.K-th Smallest Element

main.py

Share Run Output

```python
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k - 1]

    medians = []

    for i in range(0, len(arr), 5):
        group = sorted(arr[i:i + 5])
        medians.append(group[len(group) // 2])

    pivot = median_of_medians(medians, (len(medians) + 1) //
        2)

    smaller = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    larger = [x for x in arr if x > pivot]

    if k <= len(smaller):
        return median_of_medians(smaller, k)
    elif k <= len(smaller) + len(equal):
        return pivot
    else:
        return median_of_medians(
            larger,
            k - len(smaller) - len(equal)
```

```
Test Case 1
Array: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
K = 6
K-th Smallest Element: 6

Test Case 2
Array: [23, 17, 31, 44, 55, 21, 20, 18, 19, 27]
K = 5
K-th Smallest Element: 21


=== Code Execution Successful ===
```

13.Meet in the Middle – Closest Subset Sum

main.py Share Run Output

```python
from itertools import combinations
from bisect import bisect_left

def subset_sums(arr):
    sums = []

    for r in range(len(arr) + 1):
        for comb in combinations(arr, r):
            sums.append((sum(comb), list(comb)))

    return sums

def meet_in_middle(arr, target):
    mid = len(arr) // 2

    left = arr[:mid]
    right = arr[mid:]

    left_sums = subset_sums(left)
    right_sums = subset_sums(right)

    right_sums.sort()

    best_sum = 0
    best_subset = []
    best diff = float('inf')
```

```
Test Case 1
Set: [45, 34, 4, 12, 5, 2]
Target: 42
Closest Subset: [34, 5, 2]
Subset Sum: 41
Difference: 1

Test Case 2
Set: [1, 3, 2, 7, 4, 6]
Target: 10
Closest Subset: [4, 6]
Subset Sum: 10
Difference: 0


=== Code Execution Successful ===
```

14.Meet in the Middle – Exact Subset Sum

main.py Share Run Output

```python
from itertools import combinations

def subset_sums(arr):
    sums = set()

    for r in range(len(arr) + 1):
        for comb in combinations(arr, r):
            sums.add(sum(comb))

    return sums

def meet_in_middle(arr, target):
    mid = len(arr) // 2

    left = arr[:mid]
    right = arr[mid:]

    left_sums = subset_sums(left)
    right_sums = subset_sums(right)

    for s in left_sums:
        if target - s in right_sums:
            return True

    return False

```

```
Test Case 1
Array: [1, 3, 9, 2, 7, 12]
Exact Sum: 15
Output: True

Test Case 2
Array: [3, 34, 4, 12, 5, 2]
Exact Sum: 15
Output: True


=== Code Execution Successful ===
```

15.Strassen's Matrix Multiplication

main.py | Share | Run | Output

```python
def strassen(A, B):
    a = A[0][0]
    b = A[0][1]
    c = A[1][0]
    d = A[1][1]

    e = B[0][0]
    f = B[0][1]
    g = B[1][0]
    h = B[1][1]

    M1 = (a + d) * (e + h)
    M2 = (c + d) * e
    M3 = a * (f - h)
    M4 = d * (g - e)
    M5 = (a + b) * h
    M6 = (c - a) * (e + f)
    M7 = (b - d) * (g + h)

    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6

    return [[C11, C12], [C21, C22]]

```

```
Matrix A:
[1, 7]
[3, 5]
Matrix B:
[6, 8]
[4, 2]
Product Matrix C:
[34, 22]
[38, 34]

=== Code Execution Successful ===
```

16.Karatsuba Multiplication

main.py | Share | Run | Output

```python
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    p = 10 ** m

    a = x // p
    b = x % p
    c = y // p
    d = y % p

    ac = karatsuba(a, c)
    bd = karatsuba(b, d)
    abcd = karatsuba(a + b, c + d)

    ad_bc = abcd - ac - bd

    return ac * p * p + ad_bc * p + bd


x = 1234
y = 5678

```

```
X = 1234
Y = 5678
Product = 7006652

=== Code Execution Successful ===
```